

Analysis: Shifting Paths

For the small dataset, we can directly simulate the process of walking through the forest. How many steps can this take? There are 2^{10} states the trees in the clearings can be in, and 10 possible states for the clearing we are standing in. So if we reach the final clearing, it can't take more than 10×2^{10} steps, and if we take that many steps without reaching the final clearing we know the answer is Infinity.

For the large dataset, 40×2^{40} steps is too many to simulate directly, but can an input case approach this many?

A test case can be designed to make the clearings simulate an $N-1$ bit counter. The first clearing has both paths leading to the second clearing. Each clearing after that forms a chain where one path leads to the next clearing, and one path leads back to the first clearing. Whenever the trail leads back to the first clearing, the states of the clearings give the next $N-1$ bit number, and after all of those numbers have been produced, it can reach the final clearing. This will take at least 2^{39} steps.

So we need a solution that will not simulate every step individually. In the previous example, we spend $2^{21}-2$ steps in the first 20 clearings between each time we visit any of the last 20 clearings. If our program could detect that this always happens, it could simulate those $2^{21}-2$ steps without having to do them all individually. We will only take $2^{20}-3$ steps from any of the second twenty clearings, so the total runtime would be reasonable.

An implementation of this approach is to split the clearings into two sets A and B of roughly equal size. We then use dynamic programming to compute, for each location in A and each of the $2^{|A|}$ states of the clearings in A, how many steps it will take to leave A, which clearing in B we will be leaving towards, and what state the clearings in A will be left in.

After this DP is done, our solution will take an amount of time proportional to the number of steps we take from clearings in B -- for each of those, we simulate that step, and if that takes us to a clearing in A, we look up the appropriate result of the DP from a table. So to make this solution efficient, we need to choose A and B so that only a small number of steps are taken from clearings in B.

Find the distance from each clearing to the final clearing, where distance is defined as the number of paths you need to take, assuming the state of the clearings is optimal. If the final clearing cannot be reached from some clearings, put those in a separate set. If we ever reach one of those, the answer is Infinity.

Choose B to be the closest half of the remaining clearings. There can be at most $B \times 2^{|B|}$ steps taken from clearings in B, because we cannot repeat a state of B without reaching the final clearing. This can be only at most 20×2^{20} , so this solution is sufficiently fast.

Analysis: Twirling Towards Freedom

The Small Input

On most Google Code Jam problems, the small input can be solved without worrying about the running time. That was not the case here though. Even if $N = 10$ and $M = 10$, there are still 10^{10} different rotation patterns you could try. That is a lot!

There are various ways you could try to bring the running time down to a more manageable amount, but here is one fact that makes it easy:

- The 1st star you rotate around, the 5th star you rotate around, and the 9th star you rotate around should all be the same.
- The 2nd star you rotate around, the 6th star you rotate around, and the 10th star you rotate around should all be the same.
- The 3rd star you rotate around and the 7th star you rotate around should be the same.
- The 4th star you rotate around and the 8th star you rotate around should be the same.

If you realize this, there are only 10^4 possibilities to try. But why is this fact true? Read on and you'll find out!

Understanding Rotations

One of the biggest challenges here is just wrapping your head around the problem. How can you intuitively understand what it means to rotate something 100,000 times? The best way to do this is to roll up your sleeves and write down a couple formulas.

When dealing with rotations and translations of points on a plane, *complex numbers* provide an excellent notation:

- A point (x, y) can be represented as $x + iy$.
- Complex numbers can be added and subtracted just like vectors.
- Complex numbers can be rotated clockwise 90 degrees about the origin simply by multiplying them by $-i$.

It is this last property that makes them so clean to work with, and it is what we will use for the analysis. If you prefer, you could also imagine replacing everything with matrices.

We know that rotating a point P_0 by 90 degrees about the origin will send it to $-i * P_0$. However, what happens if we rotate it about a different point Q_0 ? There is a standard formula for this situation that you may have already seen. The resulting point P_1 satisfies the following:

$$P_1 - Q_0 = -i * (P_0 - Q_0)$$

This is our previous formula applied to the fact that rotating $P_0 - Q_0$ by 90 degrees about the origin must give you $P_1 - Q_0$. (Do you see why that is true? It's really just a coordinate change.) In our case, it will be helpful to group things a little differently in the formula:

$$P_1 = -iP_0 + Q_0 * (1 - i)$$

Now, suppose we rotate P_1 by 90 degrees about another point Q_1 , then by 90 degrees about another point Q_2 , and so on. What would happen to this formula? Let's write out a few examples:

- $P_2 = -iP_1 + Q_1 * (1 - i) = -P_0 + Q_1 * (1 - i) + Q_0 * (-1 - i).$
- $P_3 = -iP_2 + Q_2 * (1 - i) = iP_0 + Q_2 * (1 - i) + Q_1 * (-1 - i) + Q_0 * (-1 + i).$
- $P_4 = -iP_3 + Q_3 * (1 - i) = P_0 + Q_3 * (1 - i) + Q_2 * (-1 - i) + Q_1 * (-1 + i) + Q_0 * (1 + i).$
- $P_5 = -iP_4 + Q_4 * (1 - i) = -iP_0 + (Q_4 + Q_0) * (1 - i) + Q_3 * (-1 - i) + Q_2 * (-1 + i) + Q_1 * (1 + i).$
- etc.

From here, it's not too hard to guess the general formula. If the origin is rotated by 90 degrees about Q_0 , then Q_1 , and so on, all the way through Q_{m-1} , then the final resulting point P_m is given by:

$$(1 - i) * (Q_{m-1} + Q_{m-5} + Q_{m-9} + \dots) + (-1 - i) * (Q_{m-2} + Q_{m-6} + Q_{m-10} + \dots) + (-1 + i) * (Q_{m-3} + Q_{m-7} + Q_{m-11} + \dots) + (1 + i) * (Q_{m-4} + Q_{m-8} + Q_{m-12} + \dots)$$

And once the formula is written down, it's not too hard to check that it always works. So anyway, is this simpler than the original problem formulation? It may look pretty complicated, but for the most part, you're now just adding points together here, and addition is easier than rotation!

Pick a Direction

We want to choose stars $Q_0, Q_1, \dots, Q_{m-1}$ in such a way as to make the following point as far away from the origin as possible:

$$(1 - i) * (Q_{m-1} + Q_{m-5} + Q_{m-9} + \dots) + (-1 - i) * (Q_{m-2} + Q_{m-6} + Q_{m-10} + \dots) + (-1 + i) * (Q_{m-3} + Q_{m-7} + Q_{m-11} + \dots) + (1 + i) * (Q_{m-4} + Q_{m-8} + Q_{m-12} + \dots)$$

There are different ways of thinking about things from here, but the key is always [convex hulls](#).

Let X be the point furthest from the origin that we can attain, and more generally, let X_v be the point that is furthest in the direction of v that we can attain. Certainly, $X = X_v$ for some v . (This is because X is surely the point that is furthest in the direction of X .) Therefore, it suffices to calculate X_v for all v , and we can then measure which of these is furthest from the origin to get our final answer.

So how do we calculate X_v for some given v ? We want $(1 - i) * Q_{m-1}$ to be as far as possible in the v direction, or equivalently, we want Q_{m-1} to be the star that is furthest in the $(1 + i)*v$ direction. Of course, the same thing is true for Q_{m-5}, Q_{m-9} , etc. We should choose the same star for all of them. (This is the fact that we used for the small input solution!) Similarly, for Q_{m-2}, Q_{m-6} , etc., we want to choose the star is furthest in the $(-1 + i)*v$ direction. In general, we want to choose stars from the original set that are as far as possible in the following directions: $(1 + i)*v, (-1 + i)*v, (-1 - i)*v, (1 - i)*v$.

We are now almost done (at least conceptually). First we find the convex hull of the stars and solve for one particular v . Now what happens when we rotate v ? For a while, nothing will change, but eventually, one of the four directions we are trying to optimize will be perpendicular to an edge of the convex hull, and as a result, the optimal star will switch to the next point on the convex hull. This simulation can be done in constant time at each step, and there will be only $O(N)$ steps since each star choice will rotate once around the convex hull.

At this point, we are done! We have found every X_v , and we can manually check each of them to see which one is best. The implementation here is tricky, but it is an example of a general technique called [rotating calipers](#). You can find plenty more examples of it online. Or you can always download solutions from our finalists!

Bonus Question!

When we generated test data for this problem, we were surprised to find that we could not come up with a case to break solutions which misread "clockwise" as "counter-clockwise" in the problem statement. In fact, there is no such case! Can you see why?

Analysis: Upstairs/Downstairs

Upstairs/Downstairs was a last-minute addition to the Finals, replacing a year-old problem proposal that had appeared on a separate contest two months before. This problem loosely mirrors an experience the author had once while on vacation. He was downstairs.

Solving this problem involves two observations and an algorithm. Before making our observations, let's start by writing out the formula for the quantity we want to minimize. p_i will represent the probability that the i^{th} activity Konstantin performs will result in Ilia being awake:

$$\begin{aligned} P(\text{woken up}) = & (1-p_0) * (1 - (1-p_1)(1-p_2)*\dots*(1 - p_K)) + \\ & p_0(1-p_1) * (1 - (1-p_2)(1-p_3)*\dots*(1 - p_K)) + \\ & p_0p_1(1-p_2) * (1 - (1-p_3)(1-p_4)*\dots*(1 - p_K)) + \\ & \dots \end{aligned}$$

Observation 1: Noisy First

For our first observation, we'll look for a reason to prefer that Konstantin perform noisier or quieter activities earlier. Intuitively, we suspect that noisier activities should come first: a strategy that keeps Ilia awake and then tries to keep him asleep seems pretty reasonable.

Looking at the structure of the formula above, we can look for differences between how p_i and p_{i+1} are used. Here are the terms in which swapping them would lead to a difference in the final result:

$$\begin{aligned} & p_0p_1\dots p_{i-1}(1-p_i) * (1 - (1-p_{i+1})(1-p_{i+2})*\dots*(1 - p_K)) + \\ & p_0p_1\dots p_{i-1}p_i(1-p_{i+1}) * (1 - (1-p_{i+2})*\dots*(1 - p_K)) \end{aligned}$$

Simplifying these two terms, we have:

$$\begin{aligned} & p_0p_1\dots p_{i-1} * [\\ & \quad (1-p_i) + p_i(1-p_{i+1}) - \\ & \quad (1-p_i)(1-p_{i+1})\dots(1-p_K) - \\ & \quad p_i(1-p_{i+1})\dots(1-p_K)] \\ = & p_0p_1\dots p_{i-1} * [1 - p_i p_{i+1} - (1 - p_{i+1})(1 - p_{i+2})\dots(1 - p_K)] \end{aligned}$$

This quantity is minimized by choosing $p_{i+1} < p_i$, which confirms the intuition that noisier activities should happen earlier. Repeatedly swapping adjacent activities like this shows that in an optimal solution, activities should be performed from noisiest to quietest.

Observation 2: Go Extreme!

Our original formula for $P(\text{woken up})$ is clearly linear in p_i for all i . This quantity therefore must be minimized by taking either the largest possible value or the smallest possible value for p_i .

Putting this observation together with the previous one, we can conclude that Konstantin should start by performing the $K-q$ noisiest activities in order from noisiest to quietest, and then should perform the q quietest activities, also in order from noisiest to quietest. We'll call the first set of activities the *prefix*, and the others the *suffix*. Note that here we're considering an activity that can be repeated c times as c different activities.

Possible Algorithms

For the Small, it's sufficient to try all values of q . There are $O(K)$ possible values, and evaluating each one naively takes $O(\sum(c_i))$ time, so the running time is $O(\sum(c_i)^2)$.

There are only two numbers we really need to keep track of for each possible suffix: the probability that Ilia will end up being woken up if he starts the suffix awake, and the probability that he will end up being woken up if he starts the suffix asleep. After precomputing those 2 values for each of the K possible suffixes, which we can do in $O(K)$ time, we can simulate each possible prefix, also in $O(K)$ time, and do a quick lookup for the corresponding suffix for each one. A little math produces the correct answer, in $O(\sum(c_i))$ time.

Another algorithm that boils down to the same thing involves treating Ilia's three states—awake, asleep, and woken up—as a 3×1 vector that can be operated on by matrices. For any activity, we can build a 3×3 [transition matrix](#), looking like this:

```
[ [p      0      0]
  [1-p    1-p    0]
  [0      p     1] ]
```

Ilia's initial state is:

```
[1
 0
 0]
```

To build the transition matrix for a series of activities, we can multiply the activities' matrices together, with the noisiest activity on the right. In this way, we can compute a 3×3 matrix corresponding to each prefix of length up to K , and one for each suffix of length up to K , in $O(K)$ time. Then it's an $O(1)$ operation to check the result for any given prefix/suffix pair: multiplying those two matrices by each other, then by Ilia's initial state. The algorithm ends up taking $O(\sum(c_i))$ time.

Note that the bolded entries of the transition matrix, above, correspond to the two numbers we cared about for each suffix in the previous algorithm.

Ternary Search Passed Test Cases, But...

Because this problem was prepared at the last minute, it didn't occur to us until during the contest to wonder whether ternary search would work for selecting q , the length of the suffix. We would have been happy with the answer either way, but we would have wanted test data that would break ternary search if the answer turned out to be "no".

It turned out that the answer was "no", and that none of our existing cases broke any ternary search that we found. Because of this, some contestants—most notably **misof**, who came third partly on the strength of the points he got from this problem—managed to submit ternary search solutions that passed our test data.

Sometimes it happens in Code Jam that contestants will come up with algorithms that don't work in general given the limits we've provided, but do work on our test data. We try to avoid situations of that sort, but they do happen. We think the consequences here weren't significant, for a few reasons: Implementing the ternary search is about as hard as implementing a correct solution, if not a little harder; the "hard part" of the problem was already done by that point; and if we'd come up with a breaking case, we'd have put it in the Small data as well as the Large. Contestants who implemented ternary search would have seen it was wrong in the Small, and

taken the time to fix it. **misof** had plenty of time separating him from the fourth-place contestant, so the top three likely would not have changed.

Why Ternary Search Fails

Ternary search seems like it should work. Indeed, the function that we're minimizing, $P(\text{woken up})$, is strictly *non-increasing* and then strictly *non-decreasing* in q . Unfortunately, that isn't quite enough: it needs to be strictly *decreasing* and then strictly *increasing*; otherwise a large constant patch will leave the ternary search unable to tell which direction it should go in.

Here's an example case that breaks ternary search in principle, and breaks a few contestants' submissions in practice:

```
2
2 200
1/2 40
1/100 400
2 200
1/2 40
99/100 400
```

The correct answer is:

Case #1: 0.863976521

Case #2: 0.863976521

In the first case, the "1/2" activities are best not performed; but because they are at the very start of the list of activities, outnumbered severely by the "1/100" activities, a standard ternary search will find no difference between the two suffix lengths it tries. It will merely consider a different number of "1/100" activities to be part of the suffix as opposed to the prefix, and leave all the "1/2" activities in place. The second case is identical, but with "99/100" activities instead of "1/100" activities, which should defeat a ternary search that happens to choose the right direction in the first case.

Paweł, i Gaweł,

At dinner after the finals, several Polish contestants were shocked to discover that the problem was not based on a poem called [Paweł, i Gaweł](#). The similarity was entirely coincidental, as the contestants should have known: Gaweł, who (according to Google Translate's translation of that page) "invented the wildest frolics", lived *downstairs*. Still, in retrospect, we wish we'd named the problem after that poem, and perhaps had Gaweł minimize the probability that Paweł would go fishing.

Analysis: Xeno-archaeology

To begin with, let's try to formalize the rules of forming the pattern of tiles. If the center is at some position $\mathbf{x}$, $\mathbf{y}$, then the red tiles are those in positions $\mathbf{x}'$, $\mathbf{y}'$ for which the number $\max(|\mathbf{x} - \mathbf{x}'|, |\mathbf{y} - \mathbf{y}'|)$ is odd, while the blue tiles are those for which this number is even. This is because the formula $\max(|\mathbf{x} - \mathbf{x}'|, |\mathbf{y} - \mathbf{y}'|) = \mathbf{C}$ for any $\mathbf{C}$ describes a square ring around $\mathbf{x}$, $\mathbf{y}$, and the rings alternate color with the parity of $\mathbf{C}$.

For the small problem, we can prove that if there exists a solution, there exists one with $|\mathbf{X}| + |\mathbf{Y}| < 202$. Thus, we can check all candidates for the center, for each one check whether all the tiles have the right colors, and output the best candidate. This will not run in time for the large data, of course, as we will have over 10^{30} candidate centers to check.

Single tile analysis

We may now assume we know the parity of $\mathbf{x}$ and $\mathbf{y}$. We will simply check all four possibilities, finding the best possible choice of the center for each of the four assumptions, and then pick the one specified by the tie-breaking rules (or output "Too damaged" if none of the four assumptions led to finding a viable center for the pattern). This makes it easier to analyse the information gained by knowing the color of a single tile. Suppose the tile at some position $\mathbf{x}'$, $\mathbf{y}'$ is, say, red. This means $\max(|\mathbf{x} - \mathbf{x}'|, |\mathbf{y} - \mathbf{y}'|)$ has to be odd. Now, we know the parities of $\mathbf{x}$, $\mathbf{y}$, $\mathbf{x}'$ and $\mathbf{y}'$, and so:

- if both $\mathbf{x} - \mathbf{x}'$ and $\mathbf{y} - \mathbf{y}'$ are odd, then any choice of a center (satisfying the parity requirements for $\mathbf{x}$ and $\mathbf{y}$) is going to fit our knowledge;
- if both $\mathbf{x} - \mathbf{x}'$ and $\mathbf{y} - \mathbf{y}'$ are even, then there is no solution satisfying the parity requirements;
- if, say, $\mathbf{x} - \mathbf{x}'$ is odd, while $\mathbf{y} - \mathbf{y}'$ is even, we have to have $|\mathbf{x} - \mathbf{x}'| \geq |\mathbf{y} - \mathbf{y}'|$

If there is any tile of the second type, we can immediately return "Too damaged" for these parity assumptions. We can ignore tiles of the first type, and now we are left only with tiles of the second type.

Note that in the third case, since the parities of $\mathbf{x} - \mathbf{x}'$ and $\mathbf{y} - \mathbf{y}'$ differ, it doesn't matter whether we use a strict inequality, as the equality case is eliminated from consideration by the parity assumptions. Thus, when considering regions defined by these inequalities, we can ignore problems related to "what happens on the edges of these regions", as - by the reasoning above - the edges will necessarily be eliminated from consideration by the parity assumptions.

The first and second cases are easy to analyse; the trick is to find out whether a solution exists (and if yes, find the best one) satisfying the set of conditions of the type $|\mathbf{x} - \mathbf{x}'| \geq/\leq |\mathbf{y} - \mathbf{y}'|$ for various $\mathbf{x}'$ and $\mathbf{y}'$. Transforming the condition $|\mathbf{x} - \mathbf{x}'| \geq |\mathbf{y} - \mathbf{y}'|$ we see it is equivalent to saying that one of the following has to hold:

- $\mathbf{x} + \mathbf{y} \geq \mathbf{x}' + \mathbf{y}'$ and $\mathbf{x} - \mathbf{y} \geq \mathbf{x}' - \mathbf{y}'$, or
- $\mathbf{x} + \mathbf{y} \leq \mathbf{x}' + \mathbf{y}'$ and $\mathbf{x} - \mathbf{y} \leq \mathbf{x}' - \mathbf{y}'$.

Dividing the plane

The lines $x + y = x_i + y_i$ and $x - y = x_i - y_i$ (which are the boundaries of the constraint-satisfaction region for the input tiles) divide the plane into at most $(N + 1)^2$ rectangles. The idea of our algorithm will be as follows:

- Iterate over the four sets of parity assumptions about the center
- Iterate over all rectangles formed by the boundary lines, and for each of them check whether it satisfies the constraints posed by all input tiles
- for each rectangle satisfying the constraints, find the best (according to the tie-resolution conditions) center candidate within it (if any)
- output the best of all center candidates found.

A fun fact is that there will be at most $N+1$ rectangles that satisfy the constraints; so we need not worry overly about the performance of the "find the optimal within the rectangle" phase (as long as it is independent of the size of the rectangle). The naive approach to the second phase is $O(N^3)$ (for each rectangle check all tiles), which with N up to a thousand and 50 testcases risks being too slow, so we'll need to speed it up a bit.

There are many ways to trim down the runtime of the constraint-checking phase for rectangles. One sample way is to process the rectangles "row-by-row", as follows: Take the set of rectangles with $A \leq x+y \leq B$, with A and B being some two adjacent boundary values. For each input tile (out of those that set any constraints on the center position), we have two areas of constraint satisfaction; but only one of them is compatible with $A \leq x+y \leq B$, because one of the areas satisfies the constraint $x+y \geq C$, while the other has $x+y \leq C$. This means that we know which area is the interesting one for this row; so we obtain a constraint on $x - y$ that has to be satisfied by all the rectangles in this row. This will be either of the form $x - y \leq D$, or $x - y \geq D$. We take the largest of the lower bounds, the largest of the upper bounds, and obtain a rectangle that we have to check. This algorithm runs in $O(N^2)$ time, which will be easily fast enough.

A more advanced algorithm (using the sweep line technique) can be used to obtain a runtime of $O(N \log N)$ runtime. We will not describe it (as it is not necessary to obtain a fast enough program with the constraints given), but we encourage the reader to figure it out.

Finding the best point within a rectangle

This was the part of the problem that seemed to cause most difficulties for our contestants. There are two cases to consider here. Let's assume our rectangle is defined by $A \leq x+y \leq B$ and $C \leq x-y \leq D$.

Let us define

$g(k, l) = \min(|k|, |l|)$ if k and l are of the same sign, 0 otherwise.

If $g(A, B) = 0$ and $g(C, D) = 0$, then the point $(0, 0)$ is within our rectangle. In this case it suffices to check the near vicinity of the origin. Specifically:

- If both x and y are supposed to be even, $(0, 0)$ is obviously the optimal solution.
- If both x and y are supposed to be odd, then the best four points, in order, are $(1, 1)$, $(1, -1)$, $(-1, 1)$ and $(-1, -1)$. If $B \geq 2$ we can take $(1, 1)$ and we're done. Otherwise, if $D \geq 2$, we take $(1, -1)$; and so on. If all the four points are infeasible, the rectangle contains no points satisfying the parity constraints.
- If, say, x is supposed to be odd, while y is even, the first eight candidates are $(1, 0)$, $(-1, 0)$, $(3, 0)$, $(1, 2)$, $(1, -2)$, $(-1, 2)$, $(-1, -2)$, $(-3, 0)$. Again, one can check that if none of them is feasible, the rectangle contains no points satisfying the parity constraints. The same happens when x is even and y is odd.

Thus, if $(0, 0)$ is within the rectangle, we can check a constant number of points and take the best feasible one of them.

When $(0, 0)$ is not within the rectangle, we first look for the smallest Manhattan distance of any point within the rectangle. It is equal to $\mathbf{M} := \max(g(\mathbf{A}, \mathbf{B}), g(\mathbf{C}, \mathbf{D}))$. As all the boundaries have parities disagreeing with the parity assumptions, the smallest Manhattan distance we can hope for is $\mathbf{M} + 1$. We now have an interval of points with Manhattan distance $\mathbf{M} + 1$ in our rectangle, the best one of them is the one with the highest $\mathbf{X}$ coordinate (out of the ones fulfilling the parity conditions). The one last special case to deal with here is when the interval contains only one point, and it has the wrong parities - in this case we need to look at distance $\mathbf{M} + 3$ (the fact that one never needs to look at $\mathbf{M} + 5$ is left as an exercise).

It was also possible to solve this problem in a number of other ways. A pretty standard one was to identify a number of "suspicious points" within a rectangle (the vicinity of $(0, 0)$, the vicinity of the corners, and the vicinity of places where the coordinate axes intersect the edges of the rectangle) and check them all, taking the best solution.

Analysis: Zombie Smash

Firstly, it is worth noting that the small dataset with only 8 zombies can be solved simply by evaluating each possible permutation for the order in which to smash zombies and keeping the best one. For each permutation, simply attempt to smash the zombies in the order given, skipping any zombies that cannot be reached on time. This simple approach has exponential time complexity and clearly will not scale for the large data set.

For the more efficient approach, let us start by considering the game state represented in an inefficient way and see how it can be made more efficient. We can represent the game state at any point in time with the following tuple:

- (Time, Location, Set of zombies already smashed)

This certainly works - given this state, we essentially have the snapshot of the game at any point in time, but this is very verbose. Since we want to smash as many zombies as possible, we want to smash zombies as soon as possible, i.e. we want to arrive at the grave they pop out of as soon as possible. With that in mind, we can make the following assumption: we will arrive at the grave of the next zombie as soon as possible, and potentially stand around waiting until that zombie can be smashed. Once smashed, we will move to the next grave as quickly as we can and repeat the process. That way, it becomes unnecessary to keep track of any states where we are transiting between two graves because those states can be derived from the states at the origin and destination graves. We can change the state to be:

- (Time last zombie was smashed, Last zombie smashed, Set of zombies already smashed)

This is an improvement, but now the problem is that we keep track of all possible sets of zombies already smashed. Let's see what we can do about that.

Consider the state $(T_1, Z_1, \{Z_1\})$ where we have just smashed zombie Z_1 at time T_1 . The set of zombies already smashed contains Z_1 , and possibly a bunch of other zombies. Suppose that Z_0 is a zombie that we have smashed earlier. There are two cases: either Z_0 appears at an interval overlapping with Z_1 , or it has already appeared before Z_1 .

1. If Z_0 has already appeared before Z_1 then by T_1 it can no longer be at its grave (even if we haven't smashed it) and explicitly tracking that it has already been smashed is unnecessary.
2. Otherwise, if Z_0 appears in an interval overlapping with Z_1 , is it possible that we will attempt to smash Z_0 again, if we don't keep track of it? Suppose that Z_0 was smashed at T_0 , since the Zombie Smasher needs to recharge twice, Z_0 will be already gone because it stands around for 1000ms and it takes 1500ms for the Zombie Smasher to recharge twice. Again, it is not necessary to explicitly track the set of zombies smashed to avoid smashing the same zombie twice.

In light of the above observations we can simplify the state to be:

- (Time last zombie was smashed, Last zombie smashed, Number of zombies already smashed)

It is easy to see that we prefer earlier times over later times for smashing a zombie - the sooner we smash a zombie, the sooner we can move on to the next one, so we are only interested in

states with minimal time possible. Let us model the state transitions as a graph and minimize on time.

The game starts at time 0 and location (0, 0). Based on this information, we can generate the initial frontier of zombies that can be reached and smashed on time. Given this frontier, the times and the locations at which zombies will appear, we can apply a modified Dijkstra's Algorithm to find the set of game states that are reachable. Once we know those, we can simply return the maximum number of zombies smashed in a reachable state. Here is the pseudo-code:

```
solve():
    all_states = Q = generateStates()
    while Q is not empty:
        s = Q.popMin()
        if s.time = infinity:
            break;

        for each zombie z such that z ≠ s.zombie:
            earliest_arrival_time = s.time + max(750,
                                                dist(s.zombie, z))
            if earliest_arrival_time ≤ z.appearance + 1000:
                earliest_smash_time = max(z.appearance,
                                          earliest_arrival_time)
                Q.update(earliest_smash_time, z, s.smashed + 1)

    // Scan for states with time < infinity, keeping the maximum
    // number of zombies smashed to get the final answer.
    return best_reachable_state(all_states)

generateStates():
    states = {}
    states.Add(0, nil, 0) // Include the initial state.
    for each zombie z:
        for zombies_killed from 1 to Z:
            // For other reachable states this will be revised later.
            earliest_smash_time = infinity
            if zombies_killed = 1:
                earliest_arrival_time = dist((0, 0), z)
                if earliest_arrival_time ≤ z.appearance + 1000:
                    earliest_smash_time = max(z.appearance,
                                              earliest_arrival_time)
            states.Add(earliest_smash_time, z, zombies_killed)
    return states
```

Crude worst-case complexity analysis of the above: `generateStates()` will produce $O(Z^2)$ states as each element of the pair (Last zombie smashed, Number of zombies already smashed) can vary from 0 to Z independently. Each state will be iterated over at most once by the outer while loop of `solve()`, and the inner for loop of `solve()` will run over all zombies, costing another $O(Z)$, assuming an efficient heap is used, giving $O(Z^3)$, which is fast enough for the large dataset where $Z = 100$. Lastly, a few contestants solved this problem with dynamic programming, keeping a 2D table with zombie index in one dimension and time since that zombie has popped up in the other dimension, maximizing on the total number of zombies smashed.